Verification and Validation Report: Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

March 9, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

[symbols, abbreviations or acronyms – you can reference the SRS tables if needed —SS]

Contents

List of Tables

List of Figures

This document ...

3 Functional Requirements Evaluation

1. **test-TC1 : Physiotherapist Registration (Valid License)**

   Initial State: App launched; registration screen accessible; Firebase Auth and Firestore active.

   Input: `registerPhysio()` called with a valid license format and an active license in `validLicenses`.

   Output: Auth user created. Firestore `users/{uid}` written with `role="physio"`, `verified=true`, and `createdAt`. All assertions passed.

   Result: Pass

2. **test-TC2 : Physiotherapist Registration (Invalid License Format)**

   Initial State: App launched; registration screen accessible.

   Input: `registerPhysio()` called with an invalid license format (missing dash or incorrect digits).

   Output: Error "Invalid license format" thrown. No Auth user or Firestore writes created.

   Result: Pass

3. **test-TC3 : Physiotherapist Registration (Unverified License)**

   Initial State: App launched; registration screen accessible.

   Input: `registerPhysio()` called with valid format but license not found or `active=false` in `validLicenses`.

   Output: Error "License not verified" thrown. No Auth user or Firestore writes created.

   Result: Pass

4. **test-TC4 : Patient Registration (Valid Invite Code)**

   Initial State: App launched; `inviteCodes` collection seeded with an active, unused code containing a `physioId`.

Input: `registerPatient()` called with a valid invite code.

Output: Auth user created. Firestore `users/{uid}` written with `role="patient"` and stored `physioId`. Invite code updated with `used=true` and `usedBy={uid}`.

Result: Pass

5. **test-TC5 : Patient Registration (Non-Existent Invite Code)**

   Initial State: App launched; `inviteCodes` collection does not contain the submitted code.

   Input: `registerPatient()` called with a code that does not exist in `inviteCodes`.

   Output: Error "Invalid invite code" thrown. No Auth user or Firestore writes created.

   Result: Pass

6. **test-TC6 : Patient Registration (Inactive Invite Code)**

   Initial State: App launched; `inviteCodes` contains a matching code with `active=false`.

   Input: `registerPatient()` called with an inactive invite code.

   Output: Error "Invite code inactive" thrown. No Auth user or Firestore writes created.

   Result: Pass

7. **test-TC7 : Patient Registration (Already Used Invite Code)**

   Initial State: App launched; `inviteCodes` contains a matching code with `used=true`.

   Input: `registerPatient()` called with an already used invite code.

   Output: Error "Invite code already used" thrown. No Auth user or Firestore writes created.

   Result: Pass

8. **test-TC8 : Patient Registration (Invite Code Missing Physio Link)**

   Initial State: App launched; `inviteCodes` contains a matching code with no `physioId` field.

   Input: `registerPatient()` called with a code missing a `physioId`.

   Output: Error "Invite code missing physio link" thrown. No Auth user or Firestore writes created.

   Result: Pass

9. **test-TC9 : User Login (Valid Physiotherapist Credentials)**

   Initial State: Login screen available; Firestore contains a profile with `role="physio"`.

   Input: `loginUser()` called with valid PT credentials.

   Output: Login returns object with `uid` and profile data including `role="physio"`. No errors thrown.

   Result: Pass

10. **test-TC10 : User Login (Valid Patient Credentials)**

    Initial State: Login screen available; Firestore contains a profile with `role="patient"`.

    Input: `loginUser()` called with valid patient credentials.

    Output: Login returns object with `uid` and profile data including `role="patient"`. No errors thrown.

    Result: Pass

11. **test-TC11 : User Login (Missing Firestore Profile)**

    Initial State: Login screen available; valid Firebase Auth credentials exist but no Firestore profile.

    Input: `loginUser()` called with valid credentials.

    Output: Error "User profile missing in Firestore" thrown. Login rejected.

    Result: Pass

12. **test-TC12 : User Logout**

    Initial State: User session active.

    Input: `logoutUser()` called.

    Output: Auth sign-out called once and completes without error.

    Result: Pass

13. **test-TC13 : Get User Data**

    Initial State: Firestore populated with test user documents.

    Input: `getUserData()` called with a valid uid, a non-existent uid, and an empty uid.

    Output: Valid uid returns `UserData`. Non-existent uid returns null. Empty uid throws an error.

    Result: Pass

14. **test-TC14 : User Lookup by Email**

    Initial State: Firestore populated with test user documents.

    Input: `getUserByEmail()` called with a registered email and an unregistered email.

    Output: Registered email returns user object and true. Unregistered email returns null and false.

    Result: Pass

15. **test-TC15 : Update User Data**

    Initial State: Firestore populated with test user documents.

    Input: `updateUser()` called with a valid update payload.

    Output: Update returns true. Changes reflected in Firestore.

    Result: Pass

16. **test-TC16 : Delete User**

    Initial State: Firestore populated with test user documents.

    Input: `deleteUser()` called with a valid uid, a valid email, and a non-existent case.

Output: Valid cases return true. Non-existent case returns false.

Result: Pass

17. **test-TC17 : User Queries**

    Initial State: Firestore populated with linked PT and patient records.

    Input: `getPatientsByPhysioId()`, `searchByName()`, and `getAllUsers()` called.

    Output: Each function returns correctly filtered arrays. Errors return empty arrays.

    Result: Pass

18. **test-TC18 : User Info Lookup**

    Initial State: Firestore populated with linked PT and patient records.

    Input: `getUserdbInfo()` called with name only and with name and birthday filter.

    Output: Returns categorized patients and physiotherapists correctly. Birthday filter works as expected.

    Result: Pass

19. **test-TC19 : PT Account Delete**

    Initial State: Firestore populated with a valid physiotherapist and linked patient.

    Input: `deletePTAccount()` called with a valid physio, a non-physio user, and a non-existent patient.

    Output: Valid case deletes patient successfully. Non-physio and non-existent cases throw appropriate errors.

    Result: Pass

20. **test-TC20 : User Authentication Validation**

    Initial State: Firestore populated with test user credentials.

    Input: `usernamePwMatch()` and `authenticateUser()` called with valid credentials, an invalid password, and a non-existent user.

Output: Valid credentials return true and user object. Invalid password throws an error. Non-existent user returns null.

Result: Pass

21. **test-TC21 : System Initialization**

    Initial State: System not yet initialized.

    Input: `initializeSystem()` called.

    Output: Success message logged to console. No errors thrown.

    Result: Pass

22. **test-TC22 : Credential Validation**

    Initial State: User account exists in Firestore.

    Input: `validateCredentials()` called with a correct password and an incorrect password.

    Output: Correct password returns true. Incorrect password returns false.

    Result: Pass

23. **test-UT-4 : Account Creation (Usability)**

    Initial State: App installed on participant's device; invite code and PT license number provided.

    Input: Participant launched the app, selected their role, and completed account creation.

    Output: All participants created accounts successfully without assistance. PT license and invite code fields completed without difficulty. Minor font size feedback noted and resolved.

    Result: Pass

24. **test-UT-5 : Exercise Recording Initiation**

    Initial State: Patient logged in on mobile device; camera permission granted; backend server running.

    Input: Participant navigated to the Record screen and pressed Start Recording.

Output: Camera activated and pose detection began. On-screen prompts instructed participant to enter the detectable frame area. Navigation and button labels reported as clear.

Result: Pass

25. **test-UT-6 : Real-Time Pose Detection and Form Feedback**

    Initial State: Patient positioned within camera frame; pose detection active; backend returning joint angle and ROM metrics.

    Input: Participant performed knee extension repetitions.

    Output: App detected relevant body landmarks and displayed appropriate form feedback overlays. Rep count incremented correctly. Detection accuracy was reported as satisfactory.

    Result: Pass

26. **test-UT-7 : Exercise Page (Instructions)**

    Initial State: Patient logged in; Exercises tab accessible.

    Input: Participant navigated to the Exercises page and reviewed listed exercises.

    Output: Exercises displayed with names, target muscle groups, set/rep counts, and step-by-step instructions. Instructions reported as clear and easy to follow.

    Result: Pass

27. **test-UT-8 : Progress Screen (Session History)**

    Initial State: Patient logged in with at least one completed session stored in Firestore.

    Input: Participant navigated to the Progress tab.

    Output: Session history displayed with ROM and rep count data. Layout was readable and logically arranged. Participants suggested adding sets and pain level as future enhancements.

    Result: Pass

28. **test-UT-9 : PT Dashboard (Patient List and Activity)**

    Initial State: Physiotherapist logged in; at least one linked patient with recorded sessions.

    Input: PT participant reviewed the home dashboard and selected a patient to view their activity history.

    Output: Dashboard displayed linked patients and activity history. Patient detail view showed session history and ROM chart. Layout reported as clear and sufficient for clinical use.

    Result: Pass

29. **test-UT-10 : PT Feedback Submission**

    Initial State: Physiotherapist logged in; patient session detail page accessible.

    Input: PT participant selected a session entry and submitted a written comment.

    Output: Feedback saved to Firestore and associated with the session. Comment visible from the patient's session view. PT participants rated the feature as clinically useful.

    Result: Pass

4 Nonfunctional Requirements Evaluation

4.1 Usability

1. **UT-1 : Navigation Clarity**

   Initial State: App installed on a mobile device; user logged in.

   Input: User was asked to find and open the exercise screen, start a recording session, and return to the dashboard without assistance.

   Output: User successfully navigated through the required screens without confusion. Feedback indicated labels and buttons were clear.

   Result: Pass

2. **UT-2 : Instruction Clarity During Exercise**

   Initial State: User is on the record exercise screen with camera access enabled.

   Input: User initiated an exercise session. The application displayed live on-screen feedback instructing the user of any adjustments.

   Output: User reported that instructions were clear and prompts matched the required actions. Minor wording improvements were noted and updated.

   Result: Pass

3. **UT-3 : Survey Questionnaire – ui_testing**

   Initial State: App accessible to testers on mobile devices.

   Input: Five users completed a short usability questionnaire addressing ease of use, clarity and navigation.

   Output: Average rating across categories was $\geq 4/5$. Minor UI refinements (font size and button spacing) were applied based on feedback.

   Result: Pass

4.2 Performance

1. **PR-1 : Exercise Interface Response Time – ex_int_speed**

   Initial State: User logged in on a mobile device; backend server running.

   Input: User started recording from the record screen. Screen recording timestamps were used to measure the time from tapping "Start" to the first visible feedback/result. There were five trials conducted.

   Output: Measured response times were 1.87s, 1.92s, 1.79s, 1.95s, and 1.84s. The average end-to-end response time was 1.87 seconds ($\leq$ 2 seconds).

   Result: Pass

2. **PR-2 : Repeated Use Performance**

   Initial State: Application running normally on mobile device.

Input: User completed 10 consecutive "Start → Record Exercise → Stop" cycles.

Output: There was no significant slowdown observed across the 10 trials. Additionally, the response times were all within acceptable range and no crashes took place.

Result: Pass

4.3 Reliability / Fault Tolerance

1. **RFT-1 : Backend Failure Handling – fail_recovery**

   Initial State: User actively using the application while connected to backend.

   Input: Backend server was manually terminated during active analysis.

   Output: Application displayed a network error message and remained responsive (no crash). After backend restart, the user was able to retry and continue normal operation.

   Result: Pass

2. **RFT-2 : Network Interruption During Upload – Interrupted_vid**

   Initial State: User uploading recorded exercise video.

   Input: WiFi connection was disabled mid-upload and restored after approximately 10 seconds.

   Output: Application detected upload failure and displayed a retry option. User retried successfully and uploaded video was verified to be intact.

   Result: Pass

4.4 Security

1. **SR-1 : Unauthorized Access to Protected Endpoint**

   Initial State: Backend server running with authentication enabled.

   Input: A request was sent to a protected endpoint without authentication credentials.

Output: Server returned HTTP 401 Unauthorized. Additionally, no protected data was returned.

Result: Pass

2. **SR-2 : Invalid Credentials Login**

   Initial State: Application login screen available.

   Input: Login attempted using an email not registered in the database or incorrect password.

   Output: System denied login and displayed an appropriate error message without exposing any system details.

   Result: Pass

4.5 Maintainability and Support

1. **MS-1 : Reproducible Setup**

   Initial State: Fresh machine with no prior installation.

   Input: Project repository was cloned and installed using instructions provided in README (the frontend and backend setup).

   Output: Application ran successfully without missing dependencies. All documented setup steps were sufficient to successfully reproduce the system.

   Result: Pass

2. **MS-2 : Code Review and Quality Assurance**

   Initial State: Revision 0 completed.

   Input: Numerous peer code reviews were conducted through pull requests and issue tracking. The unit tests were executed to validate core functionality, including the metric calculations and functional logic. The UI components were reviewed for usability, clarity of instructions, and error handling. Moreover, the identified issues were documented and resolved.

Output: The core functions passed unit testing. The UI refinements were implemented based on peer feedback. The code structure maintained clear separation between modules and frontend components. Overall, there were no major functional issues identified during review.

Result: Pass

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

The following section outlines the unit tests created for all of the modules.

6.1 Authentication

6.1.1 Account Creation Module

The following tests in Table 1 verify the functionality of the account creation module for both physiotherapists and patients, including validation checks and successful user creation workflows.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC1	FR1.1, FR1.2	Register physiotherapist with valid license format and a license that exists in `validLicenses` with `active=true`.	Auth user is created. Firestore `users/{uid}` document is written with `role="physio"`, trimmed name, lowercased email, uppercased license number, `verified=true`, and `createdAt`.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC2	FR1.1, FR1.2	Register physiotherapist with an invalid license format (e.g., missing dash or incorrect digits).	Error `"Invalid license format (example: ON-123456)."` is thrown. No Auth user is created. No Firestore writes occur.	All assertions pass.	Pass
TC3	FR1.1, FR1.2	Register physiotherapist with valid format but license is not found in `validLicenses` or `active != true`.	Error `"License not verified."` is thrown. No Auth user is created. No Firestore writes occur.	All assertions pass.	Pass
TC4	FR1.1, FR1.3, FR2, FR3	Register patient with invite code that exists in `inviteCodes`, is `active=true`, `used=false`, and includes a `physioId`.	Auth user is created. Firestore `users/{uid}` document is written with `role="patient"`, trimmed name, lowercased email, stored `physioId`, normalized invite code, and `createdAt`. Invite is updated with `used=true` and `usedBy={uid}`.	All assertions pass.	Pass

Continued on next page

(Continued from previous page)

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC5	FR1.3, FR3	Register patient with an invite code that does not exist in `inviteCodes`.	Error `"Invalid invite code."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass
TC6	FR1.3, FR3	Register patient with an invite code that exists but `active=false`.	Error `"Invite code inactive."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass
TC7	FR1.3, FR2	Register patient with an invite code that exists but `used=true`.	Error `"Invite code already used."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass
TC8	FR1.3, FR2	Register patient with an invite code that exists but missing `physioId`.	Error `"Invite code missing physio link."` is thrown. No Auth user is created. No Firestore writes/updates occur.	All assertions pass.	Pass

Table 1: Account Creation Module Test Cases

6.1.2 Valid User Module

The following tests in Table 2 verify the functionality of the valid user module by ensuring that successful authentication requires an existing Firestore user

profile for both physiotherapist and patient roles.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC9	FR1, FR1.1, SR-AR1	Login with valid credentials for a physiotherapist whose Firestore profile `users/{uid}` exists and contains `role="physio"`.	Login returns an object containing the `uid` and stored profile data (including role and user fields). No errors occur.	All assertions pass.	Pass
TC10	FR1, FR1.1, SR-AR1	Login with valid credentials for a patient whose Firestore profile `users/{uid}` exists and contains `role="patient"`.	Login returns an object containing the `uid` and stored profile data (including role and user fields). No errors occur.	All assertions pass.	Pass
TC11	FR1, FR1.1, SR-AR1	Login with valid credentials where the Firestore profile `users/{uid}` does not exist.	Error `"User profile missing in Firestore."` is thrown and login is rejected.	All assertions pass.	Pass
TC12		Logout after a user session is active.	The Auth sign-out operation is called once and completes without error.	All assertions pass.	Pass

Table 2: Valid User Module Test Cases

6.2 User Account module

The tests below outline the User Account module functionality. Note that many new functions were added for this iteration. Currently, it serves as an interface between the database and the various other components. Additionally, there may be some overlap between the Account Creation module

and Valid User module. While the modules themselves perform a few different functions, they are inextricably linked due to their handling of the same entity. For example, the Valid User module consists of tests the ensure successful Login and the User Account module test cases extend that to testing for invalid and non-existent users.

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
USER MANAGEMENT FUNCTIONS					
TC13	FR23, FR21	Get User Data: Test with valid `uid`, non-existent `uid`, and empty `uid`.	Valid `uid` returns `UserData`; non-existent returns null; empty throws error.	All assertions pass.	Pass
TC14	FR21	User Lookup: Test get user by email and email existence check with registered and unregistered emails.	Registered email returns user and true; unregistered returns null and false.	All assertions pass.	Pass
TC15	FR2, FR6, FR22	Update User: Test update user data.	Valid update returns true.	All assertions pass.	Pass
TC16	FR4	Delete User: Test delete by `uid` and by email with valid and non-existent cases.	Valid cases return true; non-existent return false.	All assertions pass.	Pass
QUERY FUNCTIONS					
TC17	FR18	User Queries: Test get patients by physioId, search by name, and get all users.	Returns correctly filtered arrays; errors return empty arrays.	All assertions pass.	Pass
TC18	FR21, FR25	User Info Lookup: Test getUserdbInfo with name only and with name+birthday filter.	Returns categorized patients/physios correctly; filtering works as expected.	All assertions pass.	Pass

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
SPECIALIZED FUNCTIONS					
TC19	FR2, FR4, FR22	PT_Account_Delete: Test with valid physio, non-physio user, and non-existent patient.	Valid case deletes patient; non-physio throws error; non-existent throws error.	All assertions pass.	Pass
TC20	SR-AR1, SR-PR1	Test usernamePwMatch and authenticateUser with valid credentials, invalid password, and non-existent user.	Valid returns true/user; invalid throws appropriate errors; non-existent returns null.	All assertions pass.	Pass
TC21	SR-AR1, SR-AR2	System Initialization: Test initializeSystem.	Logs success message to console.	All assertions pass.	Pass
VALIDATION HELPERS					
TC22	FR1, SR-AR1	Test validateCredentials with correct and incorrect passwords.	Correct returns true; incorrect returns false.	All assertions pass.	Pass

Table 3: User Account Service Test Cases

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC-PA1	FR21, FR23	Test getCurrentUser with correct inputs.	Returns the correct user object.	All Assertions Pass	Pass
TC-PA2	FR21, FR23	Test getCurrentUser with no current user.	Returns null.	All Assertions Pass	Pass
TC-PA3	FR21, FR23	Test getCurrentUser with non-existent user.	Returns null.	All Assertions Pass	Pass

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC-PA4	FR21, FR23	Test getCurrentUser with Firestore error.	Returns null.	All Assertions Pass	Pass
TC-PA5	FR21, FR23	Test getCurrentUserID with correct inputs.	Returns the correct user ID.	All Assertions Pass	Pass
TC-PA6	FR21, FR23	Test getCurrentUserID with no current user.	Returns null.	All Assertions Pass	Pass
TC-PA7	FR10, FR11, FR17, FR18, FR21	Test getUserActivities with correct inputs.	Returns the correct array of activities.	All Assertions Pass	Pass
TC-PA8	FR10, FR11, FR17, FR18, FR21	Test getUserActivities with no activities.	Returns an empty array.	All Assertions Pass	Pass
TC-PA9	FR10, FR11, FR17, FR18, FR21	Test getUserActivities with Firestore error.	Returns an empty array.	All Assertions Pass	Pass
TC-PA10	FR10, FR11, FR17, FR18, FR21	Test getUserActivities with incorrect uid.	Returns an empty array.	All Assertions Pass	Pass
TC-PA11	FR10, FR11, FR17	Test getUserSummary with correct inputs.	Returns the correct summary data.	All Assertions Pass	Pass

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC-PA12	FR10, FR11, FR17	Test getUserSummary with correct inputs but duplicate dates.	Returns the correct summary data.	All Assertions Pass	Pass
TC-PA13	FR10, FR11, FR17	Test getUserSummary with Firestore error.	Returns zero summary data.	All Assertions Pass	Pass
TC-PA14	FR10, FR18	Test repsChartData with correct inputs.	Returns the correct chart data.	All Assertions Pass	Pass
TC-PA15	FR10, FR18	Test repsChartData with correct inputs and duplicate dates.	Returns the correct chart data.	All Assertions Pass	Pass
TC-PA16	FR10, FR18	Test repsChartData with activities but no reps.	Returns all 0 chart values.	All Assertions Pass	Pass
TC-PA17	FR10, FR18	Test repsChartData with no activites.	Returns all 0 chart values.	All Assertions Pass	Pass
TC-PA18	FR10, FR18	Test repsChartData with firestore error.	Returns all 0 chart values.	All Assertions Pass	Pass
TC-PA19	FR10, FR18	Test exerciseChartData with correct data.	Returns the correct chart data.	All Assertions Pass	Pass
TC-PA20	FR10, FR18	Test exerciseChartData with correct data and duplicate dates.	Returns the correct chart data.	All Assertions Pass	Pass
TC-PA21	FR10, FR18	Test exerciseChartData with no activities.	Returns all 0 chart values.	All Assertions Pass	Pass
TC-PA22	FR10, FR18	Test exerciseChartData with firestore error.	Returns all 0 chart values.	All Assertions Pass	Pass

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC-PA23	FR10, FR11, FR18, FR19, FR22	Test repsChartData with multiple users.	Returns the correct chart data for each user.	All Assertions Pass	Pass
TC-PA24	FR10, FR11, FR18, FR19, FR22	Test repsChartData with overlapping dates.	Returns the correct chart data.	All Assertions Pass	Pass
TC-PA25	FR10, FR11, FR18, FR19, FR22	Test repsChartData with non-overlapping dates.	Returns the correct chart data.	All Assertions Pass	Pass
TC-PA26	FR10, FR11, FR18, FR19, FR22	Test repsChartData with unknown user.	Returns an empty array.	All Assertions Pass	Pass
TC-PA27	FR10, FR11, FR18, FR19, FR22	Test repsChartData with invalid user ID.	Returns an empty array.	All Assertions Pass	Pass
TC-PA28	FR10, FR11, FR18, FR19, FR22	Test repsChartData with null user ID.	Returns an empty array.	All Assertions Pass	Pass

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC-PA29	FR18, FR21	Test repsChartData with null data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA30	FR18, FR21	Test repsChartData with undefined data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA31	FR18, FR21	Test repsChartData with empty data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA32	FR18, FR21	Test repsChartData with incomplete data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA33	FR17, FR18, FR19, FR22	Test repsChartData with inconsistent data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA34	FR17, FR18, FR19, FR22	Test repsChartData with corrupted data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA35	FR17, FR18, FR19, FR22	Test repsChartData with malicious data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA36	FR17, FR18, FR19, FR22	Test repsChartData with unknown user.	Returns an empty array.	All Assertions Pass	Pass
TC-PA37	FR17, FR18, FR19, FR22	Test repsChartData with invalid user ID.	Returns an empty array.	All Assertions Pass	Pass

Continued on next page

Continued from previous page

ID	Ref. Req.	Action	Expected Result	Actual Result	Result
TC-PA38	FR17, FR18, FR19, FR22	Test repsChartData with null user ID.	Returns an empty array.	All Assertions Pass	Pass
TC-PA39	FR17, FR18, FR19, FR22	Test repsChartData with null data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA40	FR2, FR3	Test repsChartData with undefined data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA41	FR2, FR3	Test repsChartData with empty data.	Returns an empty array.	All Assertions Pass	Pass
TC-PA42	FR2, FR3	Test repsChartData with incomplete data.	Returns an empty array.	All Assertions Pass	Pass

Table 4: Profile Activity Module Test Cases

7 Changes Due to Testing

There were a few changes that were implemented as a result of testing. Note that some of these changes are in progress.

7.1 Usability Testing Results

Analyses from usability testing revealed important user experience insights which has provided clarity with what should be the focus for further development. A subset of recommendations will be developed further, while others will be tabled as out of scope due to project timeline constraints. There were a total of 5 participants in the study, with 2 being PTs, and the rest were those who satisfied the pool criteria. The diverse perspectives proved to be essential by honing in on elements that may provide substantial added value to the application. Both user flows (PT and patient) were tested with the

corresponding participants.
The physiotherapist participants had 2 recommendations: A more comprehensive exercise activity record (eg. adding sets in addition to reps) for a clearer picture on the patient's performance, and inclusion of a post-exercise feedback form to provide an insight into performance. (eg. modifying the rep count due to patient's increased pain).
The other participants highlighted the need for audio cues while performing the exercise. Reading the instructions from the acceptable distance necessary for proper application function was troublesome. Secondly, additional data on the Progress tab would be beneficial for patients to keep track, in a non-technical context, of their recovery.
See Usability Report for additional information.
Note: The usability testing has been completed, but the document's Results and Discussion still remain.

[This section should highlight how feedback from the users and from the supervisor (when one exists) shaped the final product. In particular the feedback from the Rev 0 demo to the supervisor (or to potential users) should be highlighted. —SS]

8 Automated Testing

In terms of automated testing, all of the tests for the Python backend as well as the individual modules on the TypeScript frontend of the mobile application are automated. The Python tests are run using `Pytest` by simply typing `pytest` in the command line in the correct root project directory. These tests validate core backend functionality such as pose metric calculations and helper functions used during exercise processing. All the TypeScript tests can be run similarly, though they run with `Jest` and through running the following command: `npm run test`. Ultimately, these tests focus on validating frontend logic including authentication and user account management, user activity tracking, exercise data handling, performance statistics generation, and the feedback module that displays live feedback to users and allows both the physiotherapistss and patients the opopruntity to leave comments regarding the user's respective session. It is important to note that the Firebase services and camera components are mocked during testing so that we are able to separately test the logic functionality. Also, the results

for both the backend and frontend automated test suites are printed to the console, ultimately allowing us to be able to quickly identify both the passing and failing tests immediately.

9 Trace to Requirements

This section maps the tests performed to the requirements they validate, providing traceability between verification activities and project requirements.

Table 5: Functional Requirements and Corresponding Test Sections

Test Section	**Functional Requirement(s)**
Account Creation Module Tests (TC1–TC3)	FR1.1, FR1.2
Patient Registration Tests (TC4–TC8)	FR1.1, FR1.3, FR2, FR3
User Authentication Tests (TC9–TC11)	FR1, FR1.1
User Account Tests	
User Management Functions (TC13–TC16)	FR2, FR4, FR6, FR21, FR22, FR23
User Query Function (TC17–TC18)	FR18, FR21, FR25
Specialized functions (TC19–TC21)	FR2, FR4, FR22, SR-AR1, SR-AR2, SR-PR1
Validation helpers (TC22)	FR1, SR-AR1

Table 5 shows the functional requirements and their corresponding test sections, ensuring all relevant requirements have been properly tested.

Table 6: Look & Feel Tests and Corresponding Requirements

Test ID	**Non-Functional Requirement**
UT-1	LFR-AP1

Table 6 maps the Look & Feel test cases to their corresponding non-functional requirements.

Table 7: Usability & Humanity Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
UT-2	UHR-EOU2, UHR-LRN1
UT-3	UHR-EOU1, LFR-ST1

The usability and humanity requirements and their corresponding test cases are shown in Table 7.

Table 8: Performance Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
PR-1	PR-SL1, PR-SL2
PR-2	PR-SL1

Performance requirements and their test cases are outlined in Table 8.

Table 9: Robustness & Fault-Tolerance Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
RFT-1	PR-RFT1
RFT-2	PR-RFT2

Table 9 shows the robustness and fault-tolerance requirements along with their test cases.

Table 10: Operational & Environmental Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
MS-1	OER-WE1, OER-PR1

Table 10 shows the operational and environmental requirements along with their corresponding test cases.

Table 11: Security Tests and Corresponding Requirements

Test ID	Non-Functional Requirement
SR-1	SR-AR1
SR-2	SR-AR1

Table 11 outlines the security requirements and their corresponding test cases.

10 Trace to Modules

This section maps test cases to the specific modules they validated, organized by architectural levels to ensure comprehensive verification of our system.

Table 12: Tests for Core Modules

Test ID	Module
TC1–TC8	M5 (Account Creation Module)
TC9–TC12	M6 (Valid User Module)
TC13–TC19	M1 (User Interface), M2 (Profile Activity), M5 (Account Creation), M7 (User Account Module), M8 (User Activity), M10 (Performance Statistics Generator), M13 (Feedback)
TC20–TC22	M1 (User Interface), M7 (User Account Module)
UT-1, UT-2, UT-3, PR-1, PR-2, RFT-1, RFT-2	M1 (User Interface Module)
SR-1, SR-2	M5 (Account Creation Module), M6 (Valid User Module)
MS-1, MS-2	M1 (User Interface Module), M5 (Account Creation Module), M6 (Valid User Module)

Table 12 shows the mapping between test cases and the core modules they verify.

11 Code Coverage Metrics

12 Extras

[Summary of the status of your extras. If feasible, the extras should be complete by the time of writing the VnV report. If the extras are not complete, you should summarize the plans for completing them. —SS]

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

5. Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?

Maham

1. Our team worked really well together. Everyone completed their assigned tasks on time and were available to assist others if needed. We were all communicative, cordial and on schedule.

2. At the start, we split off into completing our 'Extras'. I was responsible for the Usability Testing and I had to ensure that there were results to work with. The team members conducted the tests on participants, while I worked on analysis as the results came in. Nearer to the deadline, we realized that the due date for Extras was moved to much later, (I don't think it was in the calendar until very recently), after which all the members hopped on deck to complete their corresponding module tests. Note to self: please double check your dates.

3. The usability testing results were from our peers and our supervisors. They provided some very useful suggestions and some of those ideas are fairly straightforward to implement.

4. Our VnV report deviated a fair bit from the orginal plan. Writing the code resulted in a more detailed plan being required. In addition, our team further divided some of the modules, which also resulted in more detailed tests as well. I think I would be able to anticipate them after I have gained sufficient experience and done it a few times. Some of the basic components (eg. Account creation) will have similar implementations everywhere, and those will be easy to anticipate.

5. CI proved to be very helpful throughout. Because of continuos regression tests being performed, it was ensured that any new changes will not break the code. We did use the CI element somewhat earlier, ended up realizing that there were frequent conflicts after which it was decided that we will make more PRs regularly to minimize conflicts. I really like CI, and I see myself using it in the future a fair bit. I will have to concurrently work with others on the same project and it will not become too daunting if and when changes are needed to be made/resolve issues

Vaisnavi:

- *What went well while writing this deliverable?*

  One thing that went well while writing this deliverable was our ability to collaborate and efficiently split up the tasks and sections of this report. Specifically, in the case of the unit tests and traceability, we decided as a group for each member to derive the unit tests and traceability based on the sections of the app that they had worked on. This aided efficiency and productivity overall, and allowed the report to come together easily.

- *What pain points did you experience during this deliverable, and how did you resolve them?*

  One of the biggest pain points I experienced during this deliverable was actually writing the unit tests. Writing the tests was actually harder than I expected because mocking the Firebase database was something I had never done before. Since our authentication and valid user logic uses the Firebase database, we had to mock those database calls instead of using the real database, which took some time to learn and figure out. There was definitely a learning curve, but after looking through examples and testing different approaches, I was able to get the mocks working properly. In the end, it helped me better understand how to test code that depends on external services.

Group

- *Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?*

  Parts of this document stemmed from speaking to our clients, specifically by receiving feedback from our peers who acted as proxies for physiotherapy patients. When demonstrating our application, they interacted with the system and gave relevant, useful feedback on the usability of the interface as well as how clear and easy to grasp the real-time exercise feedback was. This mainly had an impact on the sections related to usability testing, interface design, and how feedback is presented to users, especially the requirement for clear instructions

and visible/audio cues during exercises. Moreover, the other parts of the document, such as functional requirement evaluation and technical implementation, did not come directly from speaking to our clients. These were based on the requirements defined in our SRS and the testing done by our core team, since these parts relate more to the development logic and system reliability rather than user interaction.

- *In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)*

- *Reflect on your use of CI for continuous integration testing (regression testing). Did your CI work to gatekeep your PRs? Would it have been helpful if you got your CI working earlier in the project? What are your plans to use CI for testing in future projects?*

  Our project currently uses GitHub Actions for CI, but as of right now, the workflow is limited and mainly displays the pull request activity and build checks. With our implemented unit tests for both the frontend and backend, we want to integrate these directly into the CI pipeline. Specifically, the GitHub Actions workflow could be configured to automatically run the Jest test suite for the TypeScript frontend and the Pytest test suite for the Python backend whenever a pull request is opened or updated. In this case, if any test fails, the workflow would essentially fail, which would ultimately prevent changes from being merged until the issue is resolved. With this proposed change, this would make the CI pipeline much more effective for regression testing, as it would automatically verify that new code trying to be merged in does not break existing functionality. It is important to note that it would have been helpful to set this up earlier in the project, and as a result, for future projects, we would aim to do so. Thus, overall for

future projects, we would plan to set up CI earlier and integrate our automated test suites (such as Jest and Pytest) so that the tests run automatically on every pull request and prevent merges if any tests fail.